CS5187: Vision & Image

香港城市大學（東莞）
City University of Hong Kong
(Dongguan)

Zhu Zhiyu

Tutorial 04: Image Matching and Transformations

香港城市大學（東莞）
City University of Hong Kong
(Dongguan)

Key Concepts Covered:

Lecture 06: Feature Detection, Description (SIFT), and Matching.
Lecture 06: Evaluating matches (Invariance and Discriminability).
Lecture 07: Parametric transformations (Rotation, Scale, Translation).
Lecture 07: Inverse Image Warping for alignment.

（1）Implementing and Testing Descriptor Invariance

Concept: A robust feature descriptor should not change even if the image is geometrically or photometrically transformed. We will test this by extracting features from a base image and comparing them against a rotated and scaled version of the same image.

Experiment Steps:

1.Load a base image and convert it to grayscale.
2.Apply a geometric transformation (e.g., rotation by 45° and a $0.5\times$ scale).
3.Extract SIFT keypoints and descriptors from both images.
4.Match them and calculate the percentage of successful matches to validate invariance.

（1）Implementing and Testing Descriptor Invariance

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def test_descriptor_invariance(image_path):
    # 1. Load image
    img1 = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img1 is None:
        # Create a dummy image if path is invalid for testing
        img1 = np.random.randint(0, 256, (300, 300), dtype=np.uint8)

    # 2. Transform the image (Rotate 45 degrees, Scale by 0.5)
    rows, cols = img1.shape
    M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 0.5)
    img2 = cv2.warpAffine(img1, M, (cols, rows))

    # 3. Detect and Compute SIFT
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    # 4. Match features using Brute Force Matcher
    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)
    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda x: x.distance)

    # Calculate match retention rate
    retention_rate = (len(matches) / min(len(kp1), len(kp2))) * 100
    print(f"Keypoints Image 1: {len(kp1)}")
    print(f"Keypoints Image 2: {len(kp2)}")
    print(f"Total Matches: {len(matches)}")
    print(f"Invariance Retention Rate: {retention_rate:.2f}%")

    # Draw top 50 matches
    img_matches = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None, flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

    plt.figure(figsize=(10, 5))
    plt.title("Part 1: Descriptor Invariance (Rotation & Scale)")
    plt.imshow(img_matches, cmap='gray')
    plt.axis('off')
    plt.show()

# Run the experiment
# test_descriptor_invariance('your_image.jpg')
```

（2）Image matching

Concept:
Finding similar descriptors between two different images (e.g., from different viewpoints). As discussed by standard distance thresholding can yield many false positives. We will use the **Ratio Test** to measure the distance ratio between the closest and second-closest match.

Experiment Steps:
1.Extract SIFT descriptors for two overlapping images.
2.Use K-Nearest Neighbors (KNN) matching with $k=2$.
3.Apply Lowe's ratio test: Keep the match if distance(1st) < 0.75 * distance(2nd).
4.Visualize the inlier matches.

（2）Image matching



（2）Image matching

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# PART 2: Image Matching (Lowe's Ratio Test)
# ========================================
def part2_image_matching(img1, img2):
    print("--- Running Part 2: Image Matching ---")

    # 1. Initialize SIFT detector
    sift = cv2.SIFT_create()

    # 2. Detect keypoints and compute descriptors
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    # 3. Match features using K-Nearest Neighbors (k=2)
    # Using KDTree algorithm for SIFT
    index_params = dict(algorithm=1, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(des1, des2, k=2)

    # 4. Apply Lowe's Ratio Test to filter out bad matches
    good_matches = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance:
            good_matches.append(m)

    print(f"Keypoints in Image 1: {len(kp1)}")
    print(f"Keypoints in Image 2: {len(kp2)}")
    print(f"Good Matches found: {len(good_matches)}")



    # 5. Visualize the matches
    img_matches = cv2.drawMatches(
        img1, kp1, img2, kp2, good_matches, None,
        matchColor=(0, 255, 0), singlePointColor=(255, 0, 0),
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(12, 6))
    plt.title("Part 2: SIFT Feature Matching with Lowe's Ratio Test")
    plt.imshow(img_matches, cmap='gray')
    plt.axis('off')
    plt.show()

    return kp1, kp2, good_matches

if __name__ == "__main__":
    # # Step A: Generate our test images
    # image1, image2 = generate_test_images()
    image1 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_01.jpg", cv2.IMREAD_GRAYSCALE)
    image2 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_02.jpg", cv2.IMREAD_GRAYSCALE)
    # Step B: Run Part 2 (Extract features and match)
    keypoints1, keypoints2, valid_matches = part2_image_matching(image1, image2)
```

（3）Image Warping and Alignment

Target Perspective (Image 2)



Warped Image 1 (Using Homography)



Alignment Overlay
(White/Yellow = Perfect Alignment)



香港城市大學（東莞）
City University of Hong Kong (Dongguan)

（3）Image Warping and Alignment

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# PART 2: Image Matching (Lowe's Ratio Test)
# ========================================
def part2_image_matching(img1, img2):
    print("--- Running Part 2: Image Matching ---")

    # 1. Initialize SIFT detector
    sift = cv2.SIFT_create()

    # 2. Detect keypoints and compute descriptors
    kp1, des1 = sift.detectAndCompute(img1, None)
    kp2, des2 = sift.detectAndCompute(img2, None)

    # 3. Match features using K-Nearest Neighbors (k=2)
    # Using KDTree algorithm for SIFT
    index_params = dict(algorithm=1, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(des1, des2, k=2)

    # 4. Apply Lowe's Ratio Test to filter out bad matches
    good_matches = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance:
            good_matches.append(m)

    print(f"Keypoints in Image 1: {len(kp1)}")
    print(f"Keypoints in Image 2: {len(kp2)}")
    print(f"Good Matches found: {len(good_matches)}")



    # 5. Visualize the matches
    img_matches = cv2.drawMatches(
        img1, kp1, img2, kp2, good_matches, None,
        matchColor=(0, 255, 0), singlePointColor=(255, 0, 0),
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(12, 6))
    plt.title("Part 2: SIFT Feature Matching with Lowe's Ratio Test")
    plt.imshow(img_matches, cmap='gray')
    plt.axis('off')
    plt.show()

    return kp1, kp2, good_matches

if __name__ == "__main__":
    # # Step A: Generate our test images
    # image1, image2 = generate_test_images()
    image1 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_01.jpg", cv2.IMREAD_GRAYSCALE)
    image2 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_02.jpg", cv2.IMREAD_GRAYSCALE)
    # Step B: Run Part 2 (Extract features and match)
    keypoints1, keypoints2, valid_matches = part2_image_matching(image1, image2)
```

```python
def part3_warping_alignment(img1, img2, kp1, kp2, good_matches):
    print("\n--- Running Part 3: Warping and Alignment ---")

    MIN_MATCH_COUNT = 4 # We need at least 4 points to compute a homography

    if len(good_matches) < MIN_MATCH_COUNT:
        print(f"Not enough matches are found - {len(good_matches)}/{MIN_MATCH_COUNT}")
        return

    # 1. Extract the (x, y) coordinates of the good matches
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)



    # 2. Compute the Homography matrix using RANSAC
    # RANSAC makes the algorithm robust to outlier matches (false positives)
    H_matrix, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    inliers = np.sum(mask)
    print(f"Homography computed using {inliers} inlier matches.")
    print("Homography Matrix:\n", H_matrix)

    # 3. Warp Image 1 to align with Image 2's perspective
    h, w = img2.shape
    warped_img1 = cv2.warpPerspective(img1, H_matrix, (w, h))

    # 4. Create an overlay to visually verify the alignment
    # We will put the warped Image 1 in the Red channel and Image 2 in the Green/Blue channels
    alignment_check = np.zeros((h, w, 3), dtype=np.uint8)
    alignment_check[:, :, 0] = warped_img1  # Red channel
    alignment_check[:, :, 1] = img2         # Green channel
    alignment_check[:, :, 2] = img2         # Blue channel

    # Plotting the results
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    axes[0].imshow(img2, cmap='gray')
    axes[0].set_title("Target Perspective (Image 2)")
    axes[0].axis('off')

    axes[1].imshow(warped_img1, cmap='gray')
    axes[1].set_title("Warped Image 1 (Using Homography)")
    axes[1].axis('off')

    axes[2].imshow(alignment_check)
    axes[2].set_title("Alignment Overlay\n(White/Yellow = Perfect Alignment)")
    axes[2].axis('off')

    plt.tight_layout()
    plt.show()

# ========================================
# MAIN EXECUTION BLOCK
# ========================================
if __name__ == "__main__":
    # # Step A: Generate our test images
    # image1, image2 = generate_test_images()
    image1 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_01.jpg", cv2.IMREAD_GRAYSCALE)
    image2 = cv2.imread("D:\Vision_Image\Lec&Teu_04\img_02.jpg", cv2.IMREAD_GRAYSCALE)
    # Step B: Run Part 2 (Extract features and match)
    keypoints1, keypoints2, valid_matches = part2_image_matching(image1, image2)

    # Step C: Run Part 3 (Compute homography and warp)
    part3_warping_alignment(image1, image2, keypoints1, keypoints2, valid_matches)
```